1. Introduction to AI, ML, and data

From an intended decision to a valid learning experiment

Preparation. Python variables, arrays, and basic tables. No previous machine learning or probability course is assumed.

Learning goals. Formulate a task using inputs, outputs, and an objective; distinguish rules from learned models; identify features available at prediction time; explain training, validation, and test; prepare data without leakage.

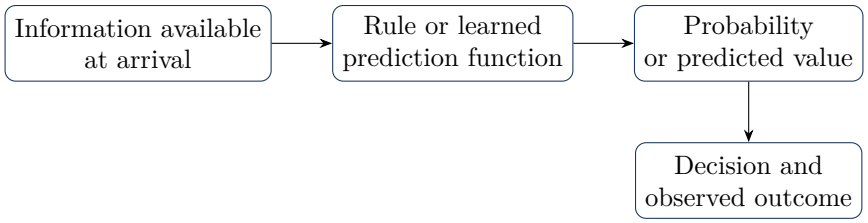
120-minute route. Problem formulation (20 min); inspect data (25); splitting and preprocessing (30); notebook demonstration (25); closing exercise (20). The companion application is a separate 60–90-minute lab.

1. Start with a problem, not an algorithm

A service receives a request. Before executing it, we would like to warn an operator if its latency is likely to exceed 110 milliseconds. A useful specification identifies *when* the decision is made, what is known then, and what a mistake costs. "Use AI to improve performance" supplies none of these details.

A rule such as "warn when the queue exceeds four requests" is designed directly by a programmer. A learned classifier instead estimates a relationship from examples. Both are computational decision procedures; neither is automatically useful because it is called AI. A fixed rule is often an informative baseline.

Artificial intelligence is a broad field concerned with computational capabilities such as reasoning, prediction, planning, and decision-making. Machine learning studies methods that improve a task-related model using data. Deep learning uses neural networks with multiple learned layers. These categories overlap. Classical search can find a path in a known map without learning from examples. A learned model can also be one component inside a planning system.



Check your understanding. If the queue rule already catches most slow requests, what evidence would justify replacing it? Name an evaluation quantity and the data on which to measure it.

Key idea. The intended use determines the prediction problem. An algorithm name does not.

2. What kind of learning problem is this?

In **supervised learning**, each example contains inputs and a target. Regression predicts a numerical quantity, such as latency in milliseconds. Classification predicts a category or its probability, such as whether a latency budget will be exceeded. The same input table can support both tasks, but their outputs, fitting losses, and useful evaluation metrics differ.

In **unsupervised learning**, no target label is supplied for each example. Grouping requests by similarity may reveal traffic types, but a cluster number does not inherently mean "slow" or "faulty." Someone must interpret the result. In **reinforcement learning**, an agent acts, receives rewards, and learns how to choose actions over time. Choosing how to schedule jobs changes what happens next; predicting their latency does not itself choose a schedule.

Task	Training information	Required output
Latency regression	Features and latency	A number in ms
Budget classification	Features and 0/1 labels	Event probability
Traffic clustering	Feature descriptions	Similarity grouping
Scheduling agent	States, actions, rewards	Policy for actions

Represent one supervised observation

Write the dataset as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i = (x_{i1}, \ldots, x_{ip}), \qquad \widehat{y}_i = f_\theta(x_i).$$

Here n is the number of rows, p the number of feature columns, and θ the fitted parameters. A data matrix has shape (n, p); the target array usually has shape $(n,)$. One row represents one request, not one feature or one user, unless the task has explicitly been defined that way.

For an arrival-time warning, payload size, queue length, cache-hit status, and endpoint type are possible inputs. Final latency is a target measurement. Putting it into the input table would let the model recover the threshold rather than predict an unknown outcome. A target can be obtained later even though it must not be used as an input earlier.

A precise specification. "For each arriving request, estimate the probability that its final latency exceeds 110 ms, using only information recorded before execution." This fixes an observation, a prediction time, inputs, and a target. It still needs a decision rule and a cost or metric before it becomes a complete application.

Check your understanding. Classify these tasks: predict tomorrow's demand; group documents by topic; choose a sequence of robot moves. Which needs labels, and which needs information about the consequences of actions?

Key idea. The learning information and required output distinguish the paradigms.

3. Inspect meaning before transforming numbers

Our recurring dataset contains independent simulated requests. The worked notebook creates 1,200 rows locally. This permits controlled, reproducible comparisons across sessions; it is not evidence about a real service.

Column	Type / unit	Meaning at prediction time
payload	Numerical / MB	Request size; some readings missing
queue	Integer / count	Number of queued requests
cache	Binary / 0 or 1	Result of an already completed cache lookup
endpoint	Category	Read or write request
violation	Target / 0 or 1	Whether eventual latency exceeds 110 ms

Questions a data audit should answer

Are units consistent? Is one payload stored in bytes and another in MB? Are there impossible negative queue lengths? Does a missing reading use a special code such as -999? Are duplicates independent events or copies of the same event? How often does the positive outcome occur? Five apparently clean rows cannot answer these questions for the complete table.

A request identifier can help trace errors while having no stable predictive meaning. A final HTTP status may look ordinary but be unavailable when a warning is needed. Feature eligibility depends on the application timeline, not on whether the field exists in a CSV. Aggregated features can also leak future information if their computation uses events that occur after the prediction time.

Missing values and categories

A missing payload does not mean zero MB. Median imputation replaces it by the median observed payload *in training rows*. This is a simple starting point; it can erase information about why a measurement is missing. A missingness indicator is a possible later feature, judged with the same evaluation discipline.

For an unordered endpoint, assigning read $= 1$ and write $= 2$ imposes numerical ordering and distance. One-hot encoding creates indicator columns instead. A future unknown category needs a handling rule. Ignoring it in an encoder prevents a program crash, but does not establish that the model understands the new category.

```
# Inspect training rows after reserving the other partitions.
X_train.dtypes
X_train.isna().sum()
X_train.describe(include="all")
y_train.value_counts(normalize=True)
```

Check your understanding. A field called mean latency per user was computed from the complete log, including tomorrow's requests. Is it legal for predicting tomorrow at midnight? How would its construction have to change?

Key idea. Preprocessing begins with semantics: what each value means and when it exists.

4. Separate learning, choosing, and evaluation

A model can memorize its training examples. Measuring performance on those same examples answers how well it fits observed data, not how well it predicts new data. Reserve observations before exploring relationships with the target.

For the independent simulation, use 60% training, 20% validation, and 20% test: 720, 240, and 240 requests. Stratification approximately preserves label proportions. Keep row indices so disjointness can be checked explicitly.

Partition	Permitted use
Training	Fit model parameters and preprocessing statistics.
Validation	Select model settings and a decision threshold.
Test	Assess frozen choices after selection.

A **parameter**, such as a slope, is fitted inside a learning algorithm. A **hyperparameter**, such as tree depth or network width, controls the model family or fitting process. Repeatedly choosing hyperparameters from test scores turns that dataset into validation, whatever its filename says.

A tiny example of information leakage

Suppose training feature values are 1 and 3, and a held-out value is 100. The training mean is 2; the full-data mean is $104/3 \approx 34.67$. Using the second number to center training data makes preparation depend on the held-out observation. Even if the effect on one model is small, the procedure no longer represents learning without access to the held-out sample.

Label leakage is more direct: a budget label defined by thresholding final latency can be predicted almost perfectly if final latency is included as an input. Such a score does not solve the arrival-time problem. A very high score should prompt an information audit, not only celebration.

A split should imitate intended use

Random splitting is reasonable for independent simulated rows. To assess next month's performance, a chronological holdout is more relevant. If many rows belong to one customer, group separation may be needed to test transfer to new customers. Stratification does not fix duplicated observations, user overlap, or future leakage.

The notebooks reuse one test partition for teaching continuity. Repeated classroom comparisons are not independent new confirmations. An actual final project should reserve a fresh, untouched evaluation set before its model search begins.

Check your understanding. An engineer tries 50 tree depths and reports only the best test score. Which role has the dataset actually played? What additional data would make the final assessment credible?

Key idea. A held-out set is defined by how it is used, not by its name.

5. Build a pipeline and establish a baseline

A pipeline groups transformations with a predictor. Calling `fit` learns from supplied training rows. Calling `transform` or `predict` reuses stored state without learning from the new labels.

For a numerical feature, standardization uses training mean μ_j and standard deviation s_j:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

Reuse these values on later rows. A library scaler handles constant features without division by zero. Scaling matters for gradient-based and distance-based methods; it is not required by ordinary threshold-based trees.

```
numeric = make_pipeline(
    SimpleImputer(strategy="median"), StandardScaler())
prep = ColumnTransformer([
    ("num", numeric, ["payload", "queue", "cache"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
     ["endpoint"]),
])
Z_train = prep.fit_transform(X_train)
Z_valid = prep.transform(X_valid)
```

This excerpt assumes imported classes and existing split tables. The worked notebook supplies a complete executable setup. The encoder may use sparse storage; that is a representation detail, not a different prediction task.

What can be achieved without useful features?

A regression baseline predicts the training target mean. A classifier can always predict the training majority class, or return the training positive fraction as a constant probability. These provide references for different metrics. They must also be fitted without validation or test labels.

Imagine 200 held-out requests with 20 violations. Always predicting "within budget" is correct on 180, giving 90% accuracy, yet catches zero violations. An alarm system may therefore need recall and false-alarm counts, which session 3 develops. A model should improve the relevant objective, not just exceed an arbitrary percentage.

The notebook experiment. Inspect training types and missingness; fit imputation, scaling, and encoding; verify that validation transformation leaves learned scales unchanged; evaluate a majority baseline on validation. This first session leaves the test unopened because no useful classifier has yet been selected.

Check your understanding. Why is `fit_transform(X_valid)` different from `transform(X_valid)`? If accuracy matches the majority baseline, what else would you inspect before judging the model?

Key idea. Data choices, transformations, fitting, decisions, and evaluation belong to one workflow.

6. Closing exercise: an experiment contract

Scenario. A helpdesk wants to predict whether a newly submitted ticket will take more than two days to resolve. Its table contains ticket text, submission hour, account identifier, final resolution time, assigned team, and a "resolved slowly" label. Team assignment occurs one hour after submission. Many tickets come from the same accounts; volume rises near exams.

1. Specify the observation, prediction time, target, and intended decision in one sentence. Distinguish the prediction from the action that follows it.
2. Identify fields unavailable at submission. Explain why assigned team depends on the exact prediction time rather than on its data type.
3. Propose a split for predicting next month's tickets. What changes if the requirement is performance on entirely new accounts?
4. State where to learn missing-value replacements and a text vocabulary. Describe what happens to a word or category seen only after training.
5. A majority baseline achieves 92% accuracy. Explain why that may be insufficient, and propose one question about its errors.
6. Write a short experiment contract: legal features, split, preprocessing, baseline, selection metric, and when the test will be opened.

A numerical check

Training queue lengths are $(0, 2, 4)$ and a validation queue length is 10. Use training mean 2 and population SD $\sqrt{8/3}$ to standardize all four values. Then calculate the mean obtained if the validation row is incorrectly included. Explain why that computation uses information from outside training.

Student application

The matching student notebook supplies the request simulation augmented with post-outcome columns. Complete three tasks: choose legal arrival-time features, fit preprocessing using only training rows, and evaluate a majority baseline on validation. Answer questions about unknown categories and future-time splits.

Submit a completed notebook with saved outputs and an explanation of what information is available when the warning is issued. A list of library calls without that explanation is incomplete. The matching application PDF contains the marking guide.

Connection to session 2. We now have inputs, a target, and an experiment. Next we ask how a numerical predictor learns coefficients by minimizing error. Before proceeding, be able to give concrete examples of a useful fixed rule, a training-perfect but useless model, and a preprocessing quantity learned from data.

Key idea. Finish by defending the experiment, not just by running a model.